SIMATS ENGINEERING

ASSESSMENT TOOL 3 - Output Prediction

COURSE NAME: Data Structures **COURSE CODE:** CSA0308

Register Number	**192521264**
Name	**Deepika shree. S**

Assessment Tool Weightage: 35%

COURSE OUTCOME COVERED

CO1: *Apply suitable data structures and ADTs for efficient problem solving by analyzing time and space complexity. (BL3, BL4)*

Questions: 5 **EACH QUESTIONS CARRIES:** 5 MARKS **Total Marks:** 25 **Duration:** ONE HOUR

S. No.	List of Questions	

1	**Array Traversal Program**	`#include <stdio.h>`
		`int main()`
		`{`
		`  int a[5] = {10, 20, 30, 40, 50};`
		`  int i;`
		`  for(i = 0; i < 5; i++)`
		`  printf("%d ", a[i]);`
		`  return 0;`
		`}`

2	**Sum of Array Elements Program**	`#include <stdio.h>`
		`int main()`
		`{`
		`  int a[] = {2, 4, 6, 8};`
		`  int sum = 0, i;`
		
		`  for(i = 0; i < 4; i++)`
		`  sum += a[i];`
		
		`  printf("%d", sum);`
		`  return 0;`
		`}`

3	**Recursive Factorial Program**	`#include <stdio.h>`
		
		`int fact(int n)`
		`{`
		`  if(n == 0)`
		`  return 1;`
		`  return n * fact(n - 1);`
		`}`
		
		`int main()`
		`{`
		`  printf("%d", fact(4));`
		`  return 0;`
		`}`

4	**Recursive Fibonacci Program**	`#include <stdio.h>`
		
		`int fib(int n)`
		`{`
		`  if(n <= 1)`
		`  return n;`
		`  return fib(n-1) + fib(n-2);`
		`}`
		
		`int main()`
		`{`
		`  printf("%d", fib(6));`
		`  return 0;`
		`}`

5	**Function as ADT Operation Program**	`#include <stdio.h>`
		
		`int add(int x, int y)`
		`{`
		`  return x + y;`
		`}`
		
		`int main()`
		`{`
		`  printf("%d", add(15, 25));`
		`  return 0;`
		`}`

Code of Conduct:

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

RUBRICS FOR CALCULATION:

Criteria	Weightage (5 Marks)	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (3 Marks)	Level 1 – Beginning (1–2 Marks)
Understanding of Program Logic	1	Correctly identifies the complete program flow and execution sequence.	Minor misunderstanding of program flow.	Partial understanding of logic.	Unable to interpret the program logic.
Trace of Variables / Function Calls	1	Accurately traces all variable values, iterations, and recursive calls.	Traces most values correctly with minor errors.	Traces some values but misses important steps.	Incorrect or no tracing.
Output Prediction Accuracy	2	Output predicted completely and correctly.	Output mostly correct with minor mistakes.	Output partially correct.	Output incorrect or missing.
Complexity Interpretation (Time/Space)	1	Correctly identifies and justifies complexity where applicable.	Correct complexity with limited justification.	Partial understanding of complexity.	Incorrect or no complexity analysis.

ANSWERS

Output Prediction with Trace and Complexity Analysis

Question 1: Array Traversal Program

Code:

```
#include <stdio.h>
Int main()
{
Int a[5] = {10, 20, 30, 40, 50};
Int I;
For(I = 0; I < 5; i++)
Printf("%d ", a[i]);
Return 0;
}
```

Answer (Output):

10 20 30 40 50

(Each number is followed by one space, so a trailing space follows 50. The output is printed on a single line.)

Reason / Explanation:

Understanding of program logic:

1. The array a[5] is initialised with the five elements 10, 20, 30, 40 and 50 stored at indices 0 to 4.

2. The for loop sets I = 0, checks I < 5 before every iteration, and increments I by 1 after each iteration.

3. In every iteration printf("%d ", a[i]) prints the element at index I followed by a space. No newline is printed.

4. When I becomes 5 the condition 5 < 5 is false, so the loop stops and the program returns 0.

Trace table:

Iteration	I	I < 5	a[i] printed	Output so far
1	0	True	10	10
2	1	True	20	10 20
3	2	True	30	10 20 30
4	3	True	40	10 20 30 40
5	4	True	50	10 20 30 40 50
Exit	5	False	-	Loop ends

Complexity analysis:

Time complexity: O(n). The loop visits each of the n = 5 elements exactly once and does a constant amount of work (one printf) per element, so T(n) = n x c = O(n).

Space complexity: O(1) auxiliary space. Only one extra variable (i) is used besides the input array. The array itself occupies O(n) space (5 integers).

Question 2: Sum of Array Elements Program

Code:

```
#include <stdio.h>
Int main()
{
Int a[] = {2, 4, 6, 8};
Int sum = 0, I;
For(I = 0; I < 4; i++)
Sum += a[i];
Printf("%d", sum);
Return 0;
}
```

Answer (Output):

20

Reason / Explanation:

Understanding of program logic:

1. The array a[] is initialised with 4 elements {2, 4, 6, 8}, so its size is 4 and the valid indices are 0 to 3.

2. Sum is initialised to 0 and acts as an accumulator.

3. The loop runs for I = 0, 1, 2, 3 and in each pass executes sum += a[i], i.e. sum = sum + a[i].

4. After the loop ends, printf("%d", sum) prints the final value of sum.

Trace table:

Iteration	I	a[i]	sum before	Operation	sum after
1	0	2	0	0 + 2	2
2	1	4	2	2 + 4	6
3	2	6	6	6 + 6	12
4	3	8	12	12 + 8	20
Exit	4	-	20	4 < 4 is False	20

Verification: 2 + 4 + 6 + 8 = 20.

Complexity analysis:

Time complexity: O(n). Each element is added exactly once (n = 4 additions), with constant work per addition.

Space complexity: O(1) auxiliary space. Only the variables sum and I are needed, regardless of the array size. The input array itself takes O(n).

Question 3: Recursive Factorial Program

Code:

```
#include <stdio.h>
Int fact(int n)
{
If(n == 0)
Return 1;
Return n * fact(n – 1);
}
Int main()
{
Printf(“%d”, fact(4));
Return 0;
}
```

Answer (Output):

24

Reason / Explanation:

Understanding of program logic:

1. Fact(n) computes n! recursively. The base case is n == 0, which returns 1. Otherwise it returns n * fact(n – 1).

2. Main() calls fact(4). Each call waits for the call below it, so calls pile up on the call stack until fact(0) is reached; then the stack unwinds and the pending multiplications are completed in reverse order.

Trace of recursive calls (winding phase – calls are made):

Step	Call	What happens
1	fact(4)	n = 4, not 0, so it computes 4 * fact(3) and waits
2	fact(3)	n = 3, not 0, so it computes 3 * fact(2) and waits
3	fact(2)	n = 2, not 0, so it computes 2 * fact(1) and waits
4	fact(1)	n = 1, not 0, so it computes 1 * fact(0) and waits
5	fact(0)	n == 0 (base case), so it returns 1

Unwinding phase (values are returned):

Step	Call	Value returned
6	fact(1)	1 * fact(0) = 1 * 1 = 1
7	fact(2)	2 * fact(1) = 2 * 1 = 2
8	fact(3)	3 * fact(2) = 3 * 2 = 6
9	fact(4)	4 * fact(3) = 4 * 6 = 24

Deepest call stack (top to bottom): fact(0), fact(1), fact(2), fact(3), fact(4), main(). Therefore fact(4) = 4 x 3 x 2 x 1 = 24, and printf prints 24.

Complexity analysis:

Time complexity: O(n). The recurrence is T(n) = T(n – 1) + O(1) with T(0) = O(1). It makes n + 1 calls (here 5 calls: fact(4) down to fact(0)), each doing constant work, so T(n) = O(n).

Space complexity: O(n). Every pending call keeps its own stack frame until the base case, so the maximum depth is n + 1 = 5 frames, i.e. O(n) stack space.

Question 4: Recursive Fibonacci Program

Code:

```
#include <stdio.h>
Int fib(int n)
{
If(n <= 1)
Return n;
Return fib(n-1) + fib(n-2);
}
Int main()
{
Printf(“%d”, fib(6));
Return 0;
}
```

Answer (Output):

8

Reason / Explanation:

Understanding of program logic:

1. Fib(n) returns the n-th Fibonacci number. The base case is n <= 1, which returns n itself (fib(0) = 0, fib(1) = 1).

2. For n >= 2 it returns fib(n-1) + fib(n-2), so every call splits into two further recursive calls and forms a binary recursion tree.

3. Main() calls fib(6) and prints the result.

Step-by-step evaluation:

1. Fib(6) = fib(5) + fib(4)
2. Fib(5) = fib(4) + fib(3)
3. Fib(4) = fib(3) + fib(2)
4. Fib(3) = fib(2) + fib(1)
5. Fib(2) = fib(1) + fib(0) = 1 + 0 = 1
6. Fib(3) = 1 + 1 = 2
7. Fib(4) = fib(3) + fib(2) = 2 + 1 = 3
8. Fib(5) = fib(4) + fib(3) = 3 + 2 = 5
9. Fib(6) = fib(5) + fib(4) = 5 + 3 = 8

Fibonacci values used:

N	0	1	2	3	4	5	6
Fib(n)	0	1	1	2	3	5	8

Recursion tree of fib(6) (each line shows a call and the value it returns):

Total number of function calls made for fib(6) = 25.

Number of times each value is recomputed:

Call	fib(6)	fib(5)	fib(4)	fib(3)	fib(2)	fib(1)	fib(0)
Times called	1	1	2	3	5	8	5

(1 + 1 + 2 + 3 + 5 + 8 + 5 = 25 calls.) The same sub-problems are solved again and again, for example fib(2) is computed 5 times.

Complexity analysis:

Time complexity: O(2^n) (exponential). The recurrence is T(n) = T(n – 1) + T(n – 2) + O(1). The call count roughly doubles with every increase of n (the exact bound is about O(1.618^n)). For n = 6 this already gives 25 calls.

Space complexity: O(n). Calls are evaluated depth-first, so only one root-to-leaf path is on the stack at a time. The deepest path is fib(6) -> fib(5) -> fib(4) -> fib(3) -> fib(2) -> fib(1), which is 6 frames, i.e. O(n).

Note: The repeated work can be removed with memoization or an iterative (dynamic programming) approach, which reduces time to O(n).

Question 5: Function as ADT Operation Program

Code:

```
#include <stdio.h>
Int add(int x, int y)
{
Return x + y;
}
Int main()
{
Printf(“%d”, add(15, 25));
Return 0;
}
```

Answer (Output):

40

Reason / Explanation:

Understanding of program logic:

1. Add(int x, int y) is a function that takes two integers and returns their sum x + y. It works as an ADT operation: the caller only needs to know what it does (its interface), not how it is implemented.

2. Main() calls add(15, 25). The value 15 is copied into x and 25 is copied into y (call by value).

3. The function evaluates x + y = 15 + 25 = 40 and returns it to main().

4. Printf("%d", …) prints the returned value 40.

Trace of the function call:

Step	Action	State
1	main() calls add(15, 25)	Control passes to add()
2	Parameters are initialised	x = 15, y = 25
3	return x + y is executed	15 + 25 = 40
4	Value is returned to main()	add(15, 25) = 40
5	printf prints the value	Output: 40

Complexity analysis:

Time complexity: O(1). The function performs a single addition and returns, which takes constant time and does not depend on the size of the input values.

Space complexity: O(1). Only the parameters x and y and one return value are used, which is a constant amount of memory.